

SDK update rules

▾ Android

[Changelog](#)[Important update for outdated versions of the AppMetrica SDK](#)[List of modules](#)[Excluding geo from the list of dependencies](#)[Disabling automatic collection of Ad Revenue data](#)[Getting advertising IDs](#)

> Migration guides

▾ AppMetrica SDK

[SDK integration](#)[Library features](#)[Tracking user activity](#)[Logging](#)[Usage examples](#)[Analyzing crashes](#)[AppMetrica Gradle Plugin](#)[Deferred deeplinks support](#)[Product flow and offer events](#)[Error descriptions](#)[SDK reference](#)

> Push SDK

> iOS

> Unity

> Flutter

> React Native

> Push campaigns

> Setting up data collection

[SDK Developer docs](#) / [Android](#) / [AppMetrica SDK](#) / [Usage examples](#)

Usage examples

Library initialization with the extended configuration

The extended configuration lets you enable/disable logging, set session timeout, pass parameters for [tracking pre-install](#) and more.

Extended configuration settings take effect immediately upon library initialization.

To initialize the library with the extended startup configuration, create an instance of the [AppMetricaConfig](#) class with settings and activate the library using the [AppMetrica.activate\(Context context, AppMetricaConfig config\)](#)

Kotlin Java

```
class YourApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        // Creating an extended library configuration.
        val config = AppMetricaConfig.newConfigBuilder(API_KEY)
        // Setting up the configuration. For example, to enable logging.
        .withLogs()
        // ...
        .build()
        // Initializing the AppMetrica SDK.
        AppMetrica.activate(applicationContext, config)
        // Automatic tracking of user activity.
        AppMetrica.enableActivityAutoTracking(this)
    }
}
```

To configure the library while the application is running, use the [AppMetrica](#) class methods.

Sending statistics to an additional API key

Sending data to an additional API key allows you to collect own statistics for these API keys. You can use this to control information for other users. For example, to provide access to statistics for analysts you can duplicate sending marketing additional API key. Thus they will only have access to the information they need.

To send data to an additional API key, you must use reporters. Just like for the main API key, you can set up an extended configuration for a reporter, send events, profile information, and data about in-app purchases. The reporter can work with AppMetrica SDK initialization.

Step 1. (Optional) Initialize a reporter with an extended configuration

To initialize a reporter with the extended configuration, create a [ReporterConfig](#) class object with the settings you need and activate the reporter using the [AppMetrica.activateReporter\(Context context, ReporterConfig config\)](#) method. The configuration is used for a reporter with the specified API key. You can set up your own configuration for each additional



The reporter with an extended configuration should be initialized before the first call to the reporter. Otherwise, reporter will be initialized without a configuration.

Kotlin Java

```
// Creating extended configuration of the reporter.
// To create it, pass a ANOTHER_API_KEY that is different from the app's API_KEY.
val reporterConfig = ReporterConfig.newConfigBuilder(ANOTHER_API_KEY)
// Setting up the configuration. For example, to enable logging.
.withLogs()
```

```
// ...
    .build()
// Initializing a reporter.
AppMetrica.activateReporter(applicationContext, reporterConfig)
```

Step 2. Configure sending data using a reporter

To send data using a reporter, get an object that implements the [IReporter](#) interface by the [AppMetrica.getReporter\(context, String apiKey\)](#) method and use the interface methods to send reports. If the reporter was not initialized with extended configuration, calling this method will initialize the reporter for the specified API key.

Example of sending an event:

Kotlin **Java**

```
AppMetrica.getReporter(applicationContext, ANOTHER_API_KEY).reportEvent("Updates installed")
```

For correct tracking, manually configure the reporters to send events about the start and the pause of the session for each activity. Use the [resumeSession\(\)](#) and [pauseSession\(\)](#) methods in the [IReporter](#) interface when implementing [onResume\(\)](#) and [onPause\(\)](#) for your Activity:

Kotlin **Java**

```
public class YourActivity : Activity() {
    override fun onResume() {
        super.onResume()
        AppMetrica.getReporter(applicationContext, ANOTHER_API_KEY).resumeSession()
    }

    override fun onPause() {
        AppMetrica.getReporter(applicationContext, ANOTHER_API_KEY).pauseSession()
        super.onPause()
    }
}
```

Tracking app crashes

Reports on app crashes are sent by default.

To disable automatic monitoring, initialize the library with the configuration in which sending crashes is disabled. To do this, pass the `false` value to the [withLocationTracking\(boolean enabled\)](#) method when creating the extended library configuration.

Kotlin **Java**

```
// Creating an extended library configuration.
val config = AppMetrica.newConfigBuilder(API_KEY)
// Disabling the data sending about the app crashes.
    .withCrashReporting(false)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

If automatic monitoring is disabled, you can manually [submit information about crashes](#).

Manually monitoring app crashes

Reports on app crashes are sent by default. To prevent event duplication when sending crashes manually, disable the [automatic monitoring of app crashes](#).

To send crash data manually, use the [AppMetrica.reportUnhandledException\(Throwable exception\)](#) method.

Kotlin **Java**

```

val previousUncaughtExceptionHandler = Thread.getDefaultUncaughtExceptionHandler()
// Creating a new handler.
val uncaughtExceptionHandler = Thread.UncaughtExceptionHandler { thread, exception ->
    try {
        // Sending a message about an app crash to the AppMetrica server.
        AppMetrica.reportUnhandledException(exception)
    } finally {
        // Sending a message about an app crash to the system handler.
        previousUncaughtExceptionHandler?.uncaughtException(thread, exception)
    }
}
// Setting the default handler.
Thread.setDefaultUncaughtExceptionHandler(uncaughtExceptionHandler)

```

Monitoring native app crashes

Reports are sent automatically on native app crashes if the library's SO files are added to the project. By default, SO files are omitted. To add them, configure the [AppMetrica Gradle Plugin](#) as follows:

build.gradle.kts **build.gradle**

```

appmetrica {
    ...
    ndk {
        enable = { applicationVariant -> true }
    }
}

```

The plugin will automatically add the appropriate dependencies. The library added by the plugin contains SO files for every platform (arm, armv7, mips, x86). Use [abi splits](#) to exclude unnecessary SO files.

If you don't use Gradle

To disable automatic monitoring, initialize the library with the configuration in which sending crashes is disabled. To do this, pass the `false` value to the `withNativeCrashReporting(boolean enabled)` method when creating the library configuration.

Kotlin **Java**

```

// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
// Disabling the data sending about the native app crashes.
    .withNativeCrashReporting(false)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)

```

[Learn more about native code.](#)

Using the library to send advertising identifiers



Advertising identifiers are sent with events by default.

For more information about managing advertising identifiers, see the [Getting advertising IDs](#).

If you want to disable the collection of tracking identifiers (for example, while waiting for user consent), you can do so during library initialization process. To do this, pass the `false` value to the `withAdvIdentifiersTracking(boolean enabled)` method when creating the configuration.

Kotlin **Java**

```
// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
// Disabling advertising identifiers tracking
    .withAdvIdentifiersTracking(false)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

After obtaining user consent, we recommend that you enable the collection of advertising identifiers while the application is running. To do this, pass `true` to the `AppMetrica.setAdvIdentifiersTracking(boolean enabled)` method.

Kotlin **Java**

```
// Enabling advertising identifiers tracking
AppMetrica.setAdvIdentifiersTracking(true)
```

Using the library to send device location



Sending device location is disabled by default.

Starting with version 5.0.0, the AppMetrica SDK is initialized with `locationTracking` disabled by default.

To enable sending it, initialize the library with the configuration where sending device location data is enabled. To do this, pass `true` value to the `withLocationTracking(boolean enabled)` method when creating an extended library configuration.

Kotlin **Java**

```
// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
// Enabling the data sending about the device location.
    .withLocationTracking(true)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

To enable sending at app runtime, use the `AppMetrica.setLocationTracking(boolean enabled)` method:

Kotlin **Java**

```
AppMetrica.setLocationTracking(true)
```

For more precise locations, add one of the following permissions to the `AndroidManifest.xml` file:

- `android.permission.ACCESS_COARSE_LOCATION` — For approximate determination.
- `android.permission.ACCESS_FINE_LOCATION` — For accurate determination.

For example:

```
<manifest>
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
  <application>...</application>
</manifest>
```

Setting location manually

Before sending custom information about the device location, make sure that reporting is enabled.

The library determines the device location on its own. To send custom data on device location, pass an instance of the `android.location.Location` class to the `AppMetrica.setLocation(Location location)` method.

Kotlin **Java**

```
// Determining the location.
val currentLocation = ...
// Setting your own location information of the device.
AppMetrica.setLocation(currentLocation)
```

To send your own device location information using the startup configuration, pass the `android.location.Location` the `withLocation(Location Location)` method when creating the extended library configuration.

Kotlin **Java**

```
// Determining the location.
val currentLocation = ...

// Creating an extended library configuration.
AppMetricaConfig config = AppMetricaConfig.newConfigBuilder(API_KEY)
// Set your own location information of the device.
    .withLocation(currentLocation)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

To resume location detection by the library, pass the `null` value to the `AppMetrica.setLocation(Location location)` and configure [delivery of device location data](#).

Kotlin **Java**

```
AppMetrica.setLocation(null)
```

Sending a custom event

To send a custom event without nested parameters, pass a short name or description of the event to the `AppMetrica.reportEvent(String eventName)` method:

Kotlin **Java**

```
AppMetrica.reportEvent("Updates installed")
```

Sending a custom event with nested parameters

The AppMetrica SDK allows you to send your own events with nested parameters that can be set in the JSON format or attributes (Map).

JSON

To pass nested event parameters in the JSON format, use the `AppMetrica.reportEvent(String eventName, String jsonValue)` method:

Kotlin **Java**

```
val eventParameters = """{"name":"Alice", "age":"18"}"""

AppMetrica.reportEvent("New person", eventParameters)
```

Map

To send a custom event message as a set of attributes (Map), use the `AppMetrica.reportEvent(String eventName, Map<String, Object> attributes)` method:

Kotlin **Java**

```
val eventParameters = mapOf("name" to "Alice", "age" to 18)
```

```
AppMetrica.reportEvent("New person", eventParameters)
```

The AppMetrica web interface displays up to five nesting levels for events. So if an event has six or more levels, only the shown in the report. You can use the [Reporting API](#) to export up to ten levels.

For more information about events, see [Events](#).

Sending an event from the WebView's JavaScript code

The AppMetrica SDK lets you send client events from JavaScript code. Initialize the sending by calling the `initWebView` method:

Kotlin **Java**

```
val webView = findViewById<WebView>(R.id.myWebView)
// do your initialization here
webView.settings.javaScriptEnabled = true
AppMetrica.initWebViewReporting(webView)
webView.loadUrl(myURL)
```

Invoke the `initWebViewReporting` method after calling the `setJavaScriptEnabled` method but before calling load

To send an event from JavaScript code, use the `reportEvent(name, value)` method in the AppMetrica interface:

Javascript

```
function buttonClicked() {
  AppMetrica.reportEvent('Button clicked!', '{}');
}
```

Arguments of the `reportEvent` method:

- `name` : A string. Can't be null or empty.
- `value` : A JSON string. Can be null.

Sending a custom error message

To send your own error message, use the following methods:

- `AppMetrica.reportError(String message, Throwable error)`
- `AppMetrica.reportError(String groupIdIdentifier, String message)`
- `AppMetrica.reportError(String groupIdIdentifier, String message, Throwable error)`

Example with groupIdIdentifier

Example with message

Sending events from the buffer

AppMetrica SDK does not send an event immediately after it occurred. The library stores event data in the buffer. The `SendEventsBuffer` method initiates sending data from the buffer and flushes it. Use the method to force sending store after passing important checkpoints of user scenarios.

Kotlin **Java**

```
AppMetrica.sendEventsBuffer()
```



If you use this method frequently, this may result in increased internet traffic and energy consumption.

Sending ProfileId

If you know the user profile ID before initializing the AppMetrica SDK, pass it before initialization (`setUserProfileID`) c initialization with the extended configuration (`withUserProfileID`).

Otherwise, a user profile is created with the ID `appmetrica_device_id`.



AppMetrica doesn't display `predefined attributes` in the web interface if `ProfileId` sending isn't configured

You can update `ProfileId` at any time by calling `setUserProfileID`:

Kotlin **Java**

```
AppMetrica.setUserProfileID("id")
```

Sending profile attributes

To send profile attributes, pass the required attributes to the instance of the `UserProfile` class and send the instance `AppMetrica.reportUserProfile(UserProfile profile)` method. To create profile attributes, use methods of the class.

Kotlin **Java**

```
// Creating the UserProfile instance.
val userProfile = UserProfile.newBuilder()
// Updating predefined attributes.
    .apply(Attribute.name().withValue("John"))
    .apply(Attribute.gender().withValue(GenderAttribute.Gender.MALE))
    .apply(Attribute.birthDate().withAge(24))
    .apply(Attribute.notificationsEnabled().withValue(false))
// Updating custom attributes.
    .apply(Attribute.customString("string_attribute").withValue("string"))
    .apply(Attribute.customNumber("number_attribute").withValue(55.0))
    .apply(Attribute.customCounter("counter_attribute").withDelta(1.0))
    .build()
// Setting the ProfileID using the method of the AppMetrica class.
AppMetrica.setUserProfileID("id")

// Sending the UserProfile instance.
AppMetrica.reportUserProfile(userProfile)
```

Sending user profile attributes with first-party data

The first-party data includes phone numbers, email addresses, and a Telegram account login. When sending this data, n and hashing occurs using SHA-256, meaning the data is not transmitted to the server and cannot be recovered from the hashes.

Kotlin **Java**

```
// Creating the UserProfile instance.
val userProfile = UserProfile.newBuilder()
// Phone numbers
    .apply(Attribute.phoneHash().withPhoneValues("+0123456789", "+9876543210"))
// Emails
    .apply(Attribute.emailHash().withEmailValues("firstemail@example@appmetrica.io", "secondemail@example.com"))
// Telegram login
    .apply(Attribute.telegramLoginHash().withTelegramLoginValues("first example login", "second example login"))
    .build()

// Sending the UserProfile instance.
AppMetrica.reportUserProfile(userProfile)
```

Sending E-commerce events

AppMetrica doesn't let you segment E-commerce events into test and non-test events. If you use the main API key for debug purchases, the test events are included in general statistics. Therefore, to debug E-commerce event sending, use a reporter to send statistics to the additional API key.

Step 1. Configure sending E-commerce events to the test API key

For different user actions, there are appropriate types of e-commerce events. To create a specific event type, use the `ecommerceEvent` class method.

The examples below show how to send specific types of events:

Opening a page

Viewing a product profile

Viewing a product page

Adding or removing an item to/from the cart

Starting and completing a purchase

Step 2. Check the test application's report

Make in-app test purchases. After a while, check the E-commerce report in the AppMetrica interface. Make sure that the report shows E-commerce events.

For more information about the report, see [E-commerce](#).

Step 3. Configure sending events to the main API key

After successful testing, configure sending E-commerce events to the main API key.

To send the `ecommerceEvent` instance to the main API key, use the `AppMetrica.reportECommerce(@NonNull ECommerceEvent)` method.

Kotlin **Java**

```
// ...
// Sending an e-commerce event.
AppMetrica.reportECommerce(ecommerceEvent)
```

Sending Revenue

AppMetrica supports the validation of purchases made using the [Google Play Billing](#) library. Validation lets you filter purchases from hacked apps. If validation is enabled and a purchase fails it, this purchase isn't shown in reports.



Local validation with a public key is used to validate purchases on Android. To enable validation, create a public key and specify it in the settings.

Step 1. Configure sending Revenue to the test API key

AppMetrica doesn't let you segment between test and non-test revenue. If you use the main API key for debugging purchases, test purchases are included in general statistics. Therefore, to debug Revenue sending, use a reporter to send statistics to the additional API key.

This section outlines the steps for sending Revenue to the additional API key:

With validation

Without validation

Step 2. Check the test application's report

Check the Revenue report in the AppMetrica report. Make sure that the number of purchases has increased.

For more information about the report, see [In-app and Ad Revenue](#).

Step 3. Configure sending Revenue to the main API key

After successful testing, have [Revenue](#) sent to the main API key.

To send the [Revenue](#) instance to the main API key, use the `AppMetrica.reportRevenue(Revenue revenue)` method.

Kotlin **Java**

```
// ...
// Sending the Revenue instance.
AppMetrica.reportRevenue(revenue)
```

Sending Ad Revenue

AppMetrica offers several options for sending ad revenue data from ad monetization and mediation services:

- [Automatic integration](#). Supported for ironSource.
- [Simplified integration](#). Supported for:
 - [Applovin MAX](#)
 - [Digital Turbine](#)
 - [Google AdMob](#)
- [Manual integration and setup](#).

Automatic integration

Data is transmitted automatically. You can disable automatic collection of data where appropriate.

ironSource

Step 1. Set up sending ad revenue data

IronSource supports automatic collection of ad revenue data. The process is managed by the `io.appmetrica.analytics:analytics-ad-revenue-ironsource-v7` module.

Activation example:

```
AppMetricaConfig config = AppMetricaConfig.newConfigBuilder(API_KEY).build();
AppMetrica.activate(context, config);
```

Step 2. Make sure that the ad revenue data is included in the reports

1. View ads in the app.
2. Make sure that the [In-app and Ad Revenue](#) report shows the same number of ad revenue events as the number of ads.

Disabling automatic collection of ad revenue data

To disable automatic collection of ad revenue data, exclude the following module from the list of dependencies:

build.gradle.kts **build.gradle**

```
configurations.configureEach {
    exclude(group = "io.appmetrica.analytics", module = "io.appmetrica.analytics-ad-revenue-ironsource-v7")
}
```

Simplified integration

Use the simplified API for sending data. It supports popular ad monetization and mediation services.

Applovin MAX

Step 1. Make sure that the SDK is activated

Activation example:

```
AppMetricaConfig config = AppMetricaConfig.newConfigBuilder(API_KEY).build();
AppMetrica.activate(context, config);
```

Step 2. Set up sending ad revenue data

1. Once the appropriate ad instance is created, set the `MaxAdRevenueListener` using the `setRevenueListener` method.
2. Send data from the `onAdRevenuePaid(MaxAd)` method to the AppMetrica SDK: call the `AppMetrica#reportExternalAdRevenue` method, then pass the parameter sent to the `MaxAd` listener and the `AppLovinSdk` instance as arguments.

App Open Ads **Banner** **Interstitial** **Native** **Rewarded**

Kotlin **Java**

```
val maxAppOpenAdView = MaxAppOpenAd( "ad-unit-ID", applicationContext)
maxAppOpenAdView.setRevenueListener {
    AppMetrica.reportExternalAdRevenue(it, AppLovinSdk.getInstance(applicationContext))
}
```

Step 3. Make sure that the ad revenue data is included in the reports

1. View ads in the app.
2. Make sure that the [In-app and Ad Revenue](#) report shows the same number of ad revenue events as the number of ad impressions.

Digital Turbine

Step 1. Make sure that the SDK is activated

Activation example:

```
AppMetricaConfig config = AppMetricaConfig.newConfigBuilder(API_KEY).build();
AppMetrica.activate(context, config);
```

Step 2. Set up sending ad revenue data

1. For required ad types (`Banner`, `Interstitial`, and `Rewarded`), set the respective listener (`BannerListener`, `InterstitialListener`, or `RewardedListener`).
2. Set up sending `impressionData` from the listener's `onShow` method to the AppMetrica SDK using the `AppMetrica#reportExternalAdRevenue()` method.

Banner **Interstitial** **Rewarded**

Kotlin **Java**

```
Banner.setBannerListener(object:BannerListener {
    //....
    override fun onShow(placementId: String, impressionData: ImpressionData) {
        AppMetrica.reportAdRevenue(impressionData)
    }
    //....
})
```

Step 3. Make sure that the ad revenue data is included in the reports

1. View ads in the app.
2. Make sure that the [In-app and Ad Revenue](#) report shows the same number of ad revenue events as the number of ad impressions.

Google AdMob

Step 1. Make sure that the SDK is activated

Activation example:

```
AppMetricaConfig config = AppMetricaConfig.newConfigBuilder(API_KEY).build();
AppMetrica.activate(context, config);
```

Step 2. Set up sending ad revenue data

App Open Ads **Banner** **Interstitial** **Native** **Rewarded** **Rewarded interstitial**

1. In the `AppOpenAdLoadCallback#onAdLoaded` method of the ad load listener, set the `OnPaidEventListener` list
2. In the `OnPaidEventListener#onPaidEvent` method, set up sending data to AppMetrica using the `AppMetrica#reportExternalAdRevenue()` method and passing the obtained `AdValue` and `AppOpenAd` instar arguments.

Kotlin **Java**

```
AppOpenAd.load(
    this,
    "ad-unit-ID",
    adRequest,
    AppOpenAd.APP_OPEN_AD_ORIENTATION_PORTRAIT,
    object : AppOpenAdLoadCallback() {
        override fun onAdLoaded(adMobAppOpenAd: AppOpenAd) {
            super.onAdLoaded(adMobAppOpenAd)
            adMobAppOpenAd.onPaidEventListener =
                OnPaidEventListener { adValue ->
                    AppMetrica.reportExternalAdRevenue(
                        adValue,
                        adMobAppOpenAd
                    )
                }
            //.....
        }
        //.....
    })
```

Step 3. Make sure that the ad revenue data is included in the reports

1. View ads in the app.
2. Make sure that the **In-app and Ad Revenue** report shows the same number of ad revenue events as the number of :

Manual integration and setup

Use this option to manually set up the data transfer from any other ad monetization service that provides impression-level data.

Step 1. Make sure that the SDK is activated

Activation example:

```
AppMetricaConfig config = AppMetricaConfig.newConfigBuilder(API_KEY).build();
AppMetrica.activate(context, config);
```

Step 2. Set up sending ad revenue data

1. Create an `AdRevenue` instance using the `AdRevenue.Builder`.

```
Map<String, String> adRevenuePayload = new HashMap<>();
adRevenuePayload.put("payload_key_1", "payload_value_1");
adRevenuePayload.put("payload_key_2", "payload_value_2");
AdRevenue adRevenue = AdRevenue.newBuilder(new BigDecimal("100.100"), Currency.getInstance
```

```

        .withAdNetwork("ad_network")
        .withAdPlacementId("ad_placement_id")
        .withAdPlacementName("ad_placement_name")
        .withAdType(AdType.NATIVE)
        .withAdUnitId("ad_unit_id")
        .withAdUnitName("ad_unit_name")
        .withPrecision("some precision")
        .withPayload(adRevenuePayload)
        .build();

```

2. Send the `Ad Revenue` instance using the `AppMetrica.reportAdRevenue(AdRevenue adRevenue)` method.

```
AppMetrica.reportAdRevenue(adRevenue);
```

Step 3. Make sure that the ad revenue data is included in the reports

1. View ads in the app.
2. Make sure that the [In-app and Ad Revenue](#) report shows the same number of ad revenue events as the number of ads shown.

Setting the session timeout duration

By default, the session timeout is 10 seconds. The minimum acceptable value for the `sessionTimeout` parameter is 10

To change the timeout, pass the value in seconds to the `withSessionTimeout(int sessionTimeout)` method when extended library configuration.

Kotlin **Java**

```

// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
    // Setting the length of the session timeout.
    .withSessionTimeout(15)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)

```

Setting the app version

By default, the app version is set in the file `build.gradle`.

To specify the app version from the code, pass the app version to the `withAppVersion(String appVersion)` method creating the extended library configuration.

Kotlin **Java**

```

// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
    // Setting the app version.
    .withAppVersion("1.13.2")
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)

```

where `1.13.2` is the app version.

Determining the library's API level (Android)

To determine the API level of the library from the application code, use the `AppMetrica.getLibraryApiLevel()` method.

Kotlin **Java**

```
val libraryApiLevel = AppMetrica.getLibraryApiLevel()
```

Determining the library's version

To determine the library version from the application code, use the `AppMetrica.getLibraryVersion()` method:

Kotlin **Java**

```
val libraryVersion = AppMetrica.getLibraryVersion()
```

Tracking installation sources

Tracking installation source in AppMetrica SDK is enabled by default.

Tracking app opens using deeplinks

To track app openings that use deeplinks, you make changes in the Activity that handles deeplinks. Tracking app opening prerequisite for correctly tracking remarketing campaigns.



To work with deeplinks, [configure them](#) in your application.

Use the `reportAppOpen()` method to track app openings:

Kotlin **Java**

```
class DeeplinkActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (savedInstanceState == null) {
            AppMetrica.reportAppOpen(this)
        }
    }

    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        AppMetrica.reportAppOpen(intent)
    }
}
```



You can add the opening of the app via the [deferred deeplink](#) on Android.

For more information, see [How do I create a remarketing campaign in AppMetrica?](#)

Requesting a deferred deeplink

To request a deferred deeplink, pass a class instance that implements the `DeferredDeeplinkListener` interface to the `AppMetrica.requestDeferredDeeplink(DeferredDeeplinkListener listener)` method. The method returns the deeplink only at the initial application start after obtaining the Google Play Install Referrer.

Kotlin **Java**

```
AppMetrica.requestDeferredDeeplink(object : DeferredDeeplinkListener {
    override fun onDeeplinkLoaded(deeplink: String) {
        Log.i("Deeplink", "deeplink = $deeplink")
    }

    override fun onError(error: DeferredDeeplinkListener.Error, referrer: String?) {
        Log.i("Deeplink", "Error: ${error.description}, unparsed referrer: $referrer")
    }
})
```

```
})
```

For more information about deferred deeplinks, see [Support for deferred deeplinks](#)

Requesting deferred deeplink parameters

To request parameters for a deferred deeplink, pass an instance of the class implementing the [DeferredDeeplinkParametersListener](#) interface to the [AppMetrica.requestDeferredDeeplinkParameters\(DeferredDeeplinkParametersListener listener\)](#) method. The method returns the deferred deeplink parameters only at the first application start after Google Play Install Referrer obtained.

Kotlin **Java**

```
AppMetrica.requestDeferredDeeplinkParameters(object : DeferredDeeplinkParametersListener {
    override fun onParametersLoaded(parameters: Map<String, String>) {
        // Processing deeplink parameters.
        for (key in parameters.keys) {
            Log.i("Deeplink params", "key: $key value: ${parameters[key]}")
        }
    }

    override fun onError(error: DeferredDeeplinkParametersListener.Error, referrer: String) {
        // Handling the error.
        Log.e("Error!", error.description)
    }
})
```

For more information about deferred deeplinks, see [Support for deferred deeplinks](#)

Tracking new users

By default, the user is counted as a new user when the app is opened for the first time. If you connect the AppMetrica SDK to an account that already has active users, you can set up tracking new and old users to get correct statistics. Configure that by initial AppMetrica SDK using the extended [AppMetricaConfig](#) configuration:

Kotlin **Java**

```
var isFirstLaunch: Boolean = false
// Implement logic to detect whether the app is opening for the first time.
// For example, you can check for files (settings, databases, and so on),
// which the app creates on its first launch.
if (conditions) {
    isFirstLaunch = true
}
// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
    .handleFirstActivationAsUpdate(!isFirstLaunch)
    .build()
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

Disabling and enabling sending statistics

If you need confirmation from the user before sending statistical data, you should initialize the library with the disabled send option. Pass the `false` value to the [withDataSendingEnabled\(boolean value\)](#) method when creating the extended configuration.

Kotlin **Java**

```
// Creating an extended library configuration.
val config = AppMetricaConfig.newConfigBuilder(API_KEY)
// Disabling sending data.
    .withDataSendingEnabled(false)
    .build()
```

```
// Initializing the AppMetrica SDK.
AppMetrica.activate(applicationContext, config)
```

After the user has confirmed that you can send statistics (for example, in the app settings or by giving their permission at app start), enable sending of statistics using the `AppMetrica.setDataSendingEnabled(true)` method:

Kotlin **Java**

```
// Checking the status of the boolean variable. It shows the user confirmation.
if (flag) {
    // Enabling sending data.
    AppMetrica.setDataSendingEnabled(true)
}
```

Alert example

You can use any text to inform users of statistics collection. For example:

This app uses the AppMetrica analytical service provided by YANDEX LLC, ulitsa Lva Tolstogo 16, Moscow, Russia 11902 referred to as Yandex) based on the [Terms of Use](#).

AppMetrica analyzes app usage data, including the device it is running on, the installation source, calculates conversion, statistics of your activity for product analytics, ad campaign analysis, and optimization, as well as for troubleshooting. Information collected in this way cannot identify you.

Depersonalized information about your use of this app collected by AppMetrica tools will be transferred to Yandex and its Yandex's server in the EU and the Russian Federation. Yandex will process this information to assess how you use the app, reports for us on our app operation, and provide other services.

Getting AppMetrica SDK IDs

To get AppMetrica SDK IDs (`DeviceId`, `DeviceIdHash`, `UUID`), use the `requestStartupParams()` method. To get `appmetrica_device_id`, make sure to request the `DeviceIdHash`.

Kotlin **Java**

```
val startupParamsCallback = object : StartupParamsCallback {
    override fun onReceive(
        result: StartupParamsCallback.Result?,
    ) {
        val deviceIdHash = result?.deviceIdHash
        // ...
    }

    override fun onRequestError(
        reason: StartupParamsCallback.Reason,
        result: StartupParamsCallback.Result?,
    ) {
        // ..
    }
}
AppMetrica.requestStartupParams(
    this,
    startupParamsCallback,
    listOf(
        StartupParamsCallback.APPMETRICA_UUID,
        StartupParamsCallback.APPMETRICA_DEVICE_ID,
        StartupParamsCallback.APPMETRICA_DEVICE_ID_HASH
    )
)
```

Delivering the device type

If you want to set the device type, specify it when creating your configuration:

Kotlin **Java**

```
class YourApplication : Application() {  
    override fun onCreate() {  
        super.onCreate()  
        // Creating an extended library configuration.  
        val config = AppMetricaConfig.newConfigBuilder(API_KEY)  
        // Defining the device type  
        .withDeviceType(PredefinedDeviceTypes.TABLET)  
        // ...  
        .build()  
        // Initializing the AppMetrica SDK.  
        AppMetrica.activate(applicationContext, config)  
        // Automatic tracking of user activity.  
        AppMetrica.enableActivityAutoTracking(this)  
    }  
}
```

If you didn't find the answer you were looking for, you can use the feedback form to submit your question. Please describe the problem in as much detail as possible. Attach a screenshot if possible.

[Contact support](#)[Suggest an improvement for documentation](#)

Was the article helpful?

 Yes

 No

[Previous](#)[← Logging](#)[Analyzing c](#)